Used Car Price Prediction using Machine Learning

A Minor Project Report

Submitted by:
Nand Kumar Kasar (6605961),
John Jaskaran Singh (6605837),
Vineetkush Pali (6605561),
Adarsh Dwivedi (6605437),
Abhishek Kumar (6605398).

Department : CSE (AIML), RCET, Bhilai | Academic Year: 2026

Certificate

This is to certify that the minor project report entitled 'Used Car Price Prediction using Machine Learning' is a bonafide work carried out by our team, in partial fulfilment of the requirements for the degree of Btech in CSE (AIML) at RCET, Bhilai (C.G.) during the academic year 2026.

The work embodied in this report has not been submitted to any other institution or university for the award of any degree or diploma.

Head of Department: Dr. Ajay Khuswaha | Date: 11/10/2026

Declaration

I hereby declare that the project report entitled 'Used Car Price Prediction using Machine Learning' is our own original work. The datasets, tools and references used in this project have been acknowledged. This report has not been submitted elsewhere for any other degree or diploma.

Acknowledgement

I express my sincere gratitude to our Technical Training tutor, for the guidance, encouragement and valuable suggestions throughout this project. I also thank the Head of the Department, Dr. Ajay Khuswaha at RCET for providing the resources and support

needed to complete this work. I am thankful to the open-source community behind Python, scikit-learn, XGBoost and Streamlit, and to the publishers of the public dataset used in this project. Finally, I thank my family and friends for their constant support.

Abstract

The price of a used car depends on many factors such as its brand and model, age, distance driven, fuel type, transmission, seller type and ownership history. Sellers and buyers often have no reliable reference price, which leads to over-pricing, under-pricing and long negotiations. This project builds an end-to-end machine learning pipeline that predicts the selling price of a used car and delivers it through an interactive web application.

A public dataset of 4,340 used-car listings with 8 attributes was used. After removing 763 duplicate rows and 82 outlier rows, 3,495 records remained for modelling. New features were engineered (brand_model and car_age), rare car models were grouped into a single category, the target price was log-transformed because it is right-skewed, and categorical columns were encoded. Three regression models were trained and compared on the same train-test split: Linear Regression (baseline), Random Forest and XGBoost. Their R-squared scores were 60.56%, 69.83% and 72.41% respectively, so XGBoost was selected as the final model. The trained model, together with the column order and dropdown maps, is saved in a pickle file and used by a Streamlit web application that takes the car details from the user and displays the estimated price.

Keywords: used car price prediction, machine learning, regression, XGBoost, feature engineering, Streamlit.

Contents

1. Introduction
2. Literature Review
3. System Requirements and Tools
4. Dataset Description
5. System Design and Methodology
6. Implementation
7. Results and Discussion
8. Testing

9. Limitations
10. Future Scope
11. Conclusion
12. References
13. Appendix

1. Introduction

1.1 Background

The used-car market is large and largely unorganised. Unlike a new car, whose price is fixed by the manufacturer, the price of a used car is decided by the seller and negotiated with the buyer. The price is influenced by many factors at the same time: how old the car is, how far it has been driven, which brand and model it is, the fuel type, whether it has a manual or automatic gearbox, how many owners it has had, and who is selling it. Because these factors interact, estimating a fair price by intuition is difficult.

Machine learning can learn the relationship between these factors and the selling price from thousands of past listings and then estimate the price of a new car in a fraction of a second.

1.2 Motivation

- Buyers need a reference price to avoid overpaying.
- Sellers need a reference price to avoid underpricing their car.
- A data-driven estimate is more consistent than guesswork.
- The project gives practical experience of the complete machine learning life cycle: data cleaning, feature engineering, model comparison, model saving and deployment through a web interface.

1.3 Problem Statement

Given the details of a used car (brand, model, manufacturing year, kilometres driven, fuel type, seller type, transmission and number of previous owners), predict its selling price in rupees. Since the output is a continuous number, this is a supervised regression problem.

1.4 Objectives

1. To clean and prepare a real-world used-car dataset for machine learning.
2. To engineer meaningful features from raw columns, in particular brand_model and car_age.
3. To train and compare three regression models: Linear Regression, Random Forest and XGBoost.
4. To select the best model using R-squared and Mean Absolute Error on unseen test data.
5. To save the trained model and use it in a user-friendly Streamlit web application.

1.5 Scope

The system predicts a price estimate for cars similar to those in the training data. It does not account for factors that are absent from the dataset, such as the exact variant, physical condition, accident history, service records or city of sale.

1.6 Organisation of the Report

Chapter 2 reviews the techniques used. Chapter 3 lists the requirements and tools. Chapter 4 describes the dataset. Chapter 5 explains the system design. Chapter 6 describes the implementation step by step. Chapter 7 presents the results. Chapter 8 covers testing, Chapters 9 and 10 discuss limitations and future scope, and Chapter 11 concludes the report.

2. Literature Review

Predicting the price of a product from its attributes is a standard regression task. Three families of techniques are commonly used on structured (tabular) data such as used-car listings.

Technique	Basic idea	Strength	Weakness
Linear Regression	Price is a weighted sum of the features	Simple, fast, easy to interpret	Cannot capture non-linear effects or interactions between features

Technique	Basic idea	Strength	Weakness
Random Forest (Breiman, 2001)	Many deep decision trees trained on random samples of the data; predictions are averaged	Handles non-linearity, robust, little tuning	Trees are independent, so errors they share are not corrected
Gradient Boosting / XGBoost (Friedman, 2001; Chen and Guestrin, 2016)	Shallow trees are built one after another, each correcting the errors of the previous ones	Usually very accurate on tabular data, built-in regularisation	More hyper-parameters; can overfit if not controlled

In general, tree-based ensemble methods tend to outperform plain linear models on tabular data in which relationships are non-linear, which is the case for car prices (for example, a car loses value quickly in its first years and more slowly later). For this reason a linear model is used here as a baseline, and Random Forest and XGBoost are compared against it.

3. System Requirements and Tools

3.1 Software requirements

Item	Details
Programming language	Python 3
Data handling	pandas, numpy
Visualisation	matplotlib, seaborn
Machine learning	scikit-learn (Linear Regression, Random Forest, train-test split, metrics), xgboost (XGBRegressor)
Model saving	pickle (Python standard library)
Web application	Streamlit
Development tools	Jupyter Notebook / VS Code, Git and GitHub

3.2 Hardware requirements

Any modern computer with at least 4 GB RAM and a standard processor is sufficient, because the dataset is small and no GPU is needed. An internet connection is needed only to install the libraries.

4. Dataset Description

The project uses a publicly available used-car dataset from Kaggle. It contains 4,340 rows and 8 columns.

Column	Description	Type
name	Full name of the car including the variant, for example Hyundai Verna 1.6 SX	Text
year	Year of manufacture	Integer
selling_price	Selling price in rupees (target variable)	Integer
km_driven	Total kilometres driven	Integer

Column	Description	Type
fuel	Fuel type, for example Petrol or Diesel	Text
seller_type	Type of seller, for example Individual or Dealer	Text
transmission	Manual or Automatic	Text
owner	Ownership history, from First Owner to Fourth and Above Owner, or Test Drive Car	Text

Initial observations

- There are no missing values in any column, so no imputation was required.
- 763 rows (about 17.6 percent) were exact duplicates.
- The selling price is right-skewed: many low-priced cars and a few very expensive ones.

5. System Design and Methodology

5.1 Architecture

The system has two phases that are connected by a saved model file.

1. **Training phase (Jupyter notebook):** the raw dataset is cleaned, features are engineered, the data is encoded and split, three models are trained and compared, and the best model is saved.
2. **Prediction phase (Streamlit web app):** the saved model is loaded, the user enters the car details, the same preprocessing is applied to that input, and the model returns the estimated price.

The saved pickle file is the link between the two phases. It stores four items: the trained model, the exact list of training columns, the list of popular cars, and a brand-to-models map used by the dropdowns.

5.2 Workflow

1. Load the dataset.
2. Check missing values and remove duplicate rows.
3. Engineer features (brand_model, car_age) and group rare cars.
4. Handle outliers.

5. Encode categorical variables.
6. Apply a log transformation to the target and split into training and test sets.
7. Train Linear Regression, Random Forest and XGBoost.
8. Evaluate and compare the models.
9. Save the best model to a pickle file.
10. Load the model in the web app and predict from user input.

5.3 Key design rule

The preprocessing applied to a user's input in the web app must be exactly the same as the preprocessing applied to the training data. If any step differs, the model receives data that looks different from what it learned from and its predictions become unreliable. For this reason the app repeats the same car-age calculation, owner encoding, rare-car grouping, one-hot encoding and column order that were used in the notebook.

6. Implementation

6.1 Data cleaning

The dataset has no missing values. Duplicate rows were detected and removed (4,340 rows reduced to 3,577). Duplicates must be removed before splitting the data, otherwise the same record could appear in both the training and the test set, making the test score look better than it really is.

6.2 Feature engineering

Brand and model. The name column contains the full listing title with the variant, so almost every value is different and it is not directly useful. The first word was taken as the brand and the second word as the model, and the two were joined into one feature called brand_model (for example, Maruti Swift). This keeps the most important price driver, which car it is, while greatly reducing the number of distinct values.

Car age. The year column was replaced by car_age, calculated as 2026 minus the manufacturing year. Price depends on how old a car is, and age is easier for the model and for people to interpret than a calendar year. The year column was then dropped, because keeping both would be redundant.

Rare car grouping. Car models that appear fewer than 15 times were grouped into a single category called Other. A model with only a few records gives the algorithm too little information and encourages overfitting. The list of retained models (top_cars) is saved so the web app can apply the same rule.

6.3 Outlier handling

Very high prices and impossible odometer readings can distort a model. Selling prices were limited to the range between the 1st and 99th percentile, and cars with 300,000 km or more were removed. A boxplot of the selling price was drawn before this step to visualise the long upper tail. In total, 82 rows were removed.

6.4 Encoding categorical variables

Ordinal encoding (owner). Ownership has a natural order, so it was converted to numbers.

Owner	Value
Test Drive Car	0
First Owner	1
Second Owner	2
Third Owner	3
Fourth and Above Owner	4

One-hot encoding. The columns brand_model, fuel, seller_type and transmission have no natural order, so each category was converted into its own 0/1 column using pandas get_dummies with drop_first set to True. Dropping the first category of each column avoids redundant columns that always add up to one. After encoding, the table had 3,495 rows and 70 columns: 69 input features and the target.

6.5 Target transformation and data split

Car prices are right-skewed, so a few expensive cars would dominate the error of a model. The target was therefore transformed with log1p (the natural logarithm of one plus the price), which makes the distribution closer to normal and makes errors relative rather than absolute. Predictions are converted back to rupees with expm1, the inverse function.

The data was split into 80 percent training data (about 2,796 rows) and 20 percent test data (about 699 rows) with random_state set to 42, so that the split is reproducible and all models are tested on exactly the same unseen rows.

6.6 Models used

Three models were trained on the same data so that the comparison is fair.

Linear Regression (baseline). Assumes the log-price is a weighted sum of the features. It is fast and interpretable but cannot represent curved relationships or interactions, such as a different effect of age for a hatchback and a luxury sedan. It is used as a reference point.

Random Forest Regressor (100 trees). A decision tree repeatedly splits the data on the feature and threshold that best separates cheap from expensive cars. A Random Forest trains many such trees on random resamples of the data and averages their predictions, which reduces the variance of a single tree.

XGBoost Regressor (200 trees, learning rate 0.1, maximum depth 5). Gradient boosting builds shallow trees one after another. Each new tree is trained to correct the errors of all previous trees, and its contribution is scaled by the learning rate. XGBoost adds regularisation and efficient handling of sparse columns, which suits the many one-hot columns in this dataset.

6.7 Evaluation metrics

- **R-squared (R2):** the proportion of the variation in car prices that the model explains. A value of 1 is a perfect fit.
- **Mean Absolute Error (MAE):** the average absolute difference between the predicted and the actual price, in rupees.

Both metrics were calculated on the real price scale: predictions were converted back from the log scale with expm1 before comparing them with the actual prices of the test set. This is a stricter test than scoring on the log scale.

6.8 Saving the model

The best model was saved with pickle together with everything the web app needs:

- model: the trained XGBoost model
- columns: the exact training column names in order
- top_cars: the list of popular brand_model values

- brand_model_map: a dictionary of brand to list of models, used for the dropdowns

The names in brand_model_map are kept exactly as they appear in the data, so that every dropdown selection matches a name in top_cars.

6.9 Web application

The web application was built with Streamlit. When the user opens the page:

1. The saved pickle file is loaded once and cached.
2. The user selects the brand; the model dropdown then shows only the models of that brand.
3. The user enters the manufacturing year, kilometres driven, fuel type, seller type, transmission and number of owners.
4. On clicking Predict Price, the app calculates car_age and the owner value, builds the brand_model name (or Other if it is not in top_cars) and creates a one-row table.
5. The table is one-hot encoded and then aligned to the saved training columns using reindex, with missing columns filled with 0. This guarantees the model receives exactly the columns and order it was trained on.
6. The model predicts the log price, expm1 converts it to rupees, and the estimated price is displayed.

7. Results and Discussion

7.1 Model comparison

Model	R-squared on test data
Linear Regression (baseline)	60.56%
Random Forest	69.83%
XGBoost	72.41%

Mean Absolute Error of the final model: Rs. [insert the MAE value printed by the notebook].

[Insert figure: bar chart of the model comparison from the notebook]

XGBoost gave the best score. It is about 11.85 percentage points better than the Linear Regression baseline and about 2.58 percentage points better than Random Forest, so it was selected as the final model.

7.2 Discussion

- **Why tree-based models beat Linear Regression:** the price depends on age, kilometres and car type together and in a non-linear way. A single set of linear weights cannot represent this, while trees can.
- **Why XGBoost did best:** it corrects the remaining errors step by step with shallow, regularised trees, which suits a small and noisy dataset better than fully grown independent trees.
- **Why no feature scaling was applied:** tree-based models are not affected by the scale of the features, and Linear Regression predictions do not change with scaling. Only three columns are non-binary, so scaling would add complexity without any benefit.
- **Why the log transformation helped:** it reduced the effect of very expensive cars and made the errors proportional to the price.

7.3 Why the accuracy is about 72 percent

An R-squared of about 72 percent is a reasonable result for this dataset, but it is not higher for several reasons:

1. The dataset is small: only 3,495 records remained after cleaning, and the model has to learn many brand and model categories from them.
2. Important information is missing: the exact variant, engine size, power, condition, accident history and city are not available, and only the first two words of the name were used.
3. Cars with fewer than 15 records are grouped as Other, which mixes cheap and expensive cars in one category.
4. The target is the seller's asking price, which is subjective and depends on negotiation and urgency.
5. The model was evaluated on a single train-test split and the hyper-parameters were chosen by hand, so there is room for improvement through cross-validation and tuning.

7.4 Application screenshots

[Insert screenshot: home page of the Streamlit application]

[Insert screenshot: example prediction output]

8. Testing

The application was tested with different inputs to check that the predictions behave sensibly. Fill in the observed output column after running each test.

Test	Input	Expected behaviour	Observed output
1	Maruti Swift, 2016, 50,000 km, Petrol, Individual, Manual, First Owner	A plausible price in lakhs of rupees	[fill in]
2	Same car, but manufactured in 2010	Price is generally lower than in Test 1	[fill in]
3	Same car as Test 1, but 150,000 km driven	Price is generally lower than in Test 1	[fill in]
4	Same car as Test 1, but Diesel	A different price; the fuel type affects the result	[fill in]
5	Change the brand in the dropdown	The model dropdown shows only the models of that brand	[fill in]
6	Kilometres driven set to 0 and to 300,000	Both values are accepted without an error	[fill in]

9. Limitations

- The data consists of historical asking prices, so the output is an estimate and not an exact market price.
- The variant, engine size, condition and accident history of the car are not used.
- The year 2026 is fixed in the code for calculating car age, and the price level reflects the period in which the data was collected.
- Very new cars, very high kilometres and very rare models are outside what the model has learned from.
- Models that appear fewer than 15 times in the data are treated as Other, so their estimates are less precise.

- The fuel, seller type and transmission options in the app are fixed lists and are not read from the model.

10. Future Scope

- Use a larger dataset and add features such as engine capacity, power, mileage, variant, city and service history.
- Use cross-validation and hyper-parameter tuning to improve and confirm the accuracy.
- Try other models such as LightGBM or CatBoost, or combine several models.
- Display a price range instead of a single value.
- Deploy the application online (for example on Streamlit Community Cloud) and retrain it periodically with fresh listings.
- Add explanations that show which features raised or lowered the predicted price.

11. Conclusion

This project built a complete machine learning solution for estimating the price of a used car, from raw data to a working web application. The data was cleaned, new features were engineered, the target was log-transformed and categorical variables were encoded. Three models were compared on the same data: Linear Regression (60.56%), Random Forest (69.83%) and XGBoost (72.41%). XGBoost performed best and was saved and deployed through a Streamlit interface that applies the same preprocessing as the training notebook. The project demonstrates the full machine learning workflow and also highlights the effect of data size, feature quality and evaluation method on model performance. With more data and richer features, the accuracy can be improved further.

12. References

1. Chen, T. and Guestrin, C., 'XGBoost: A Scalable Tree Boosting System', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
2. Breiman, L., 'Random Forests', Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
3. Friedman, J. H., 'Greedy Function Approximation: A Gradient Boosting Machine', The Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.
4. Pedregosa, F. et al., 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

5. McKinney, W., 'Data Structures for Statistical Computing in Python', Proceedings of the 9th Python in Science Conference, 2010.
6. Streamlit documentation: https://docs.streamlit.io
7. XGBoost documentation: https://xgboost.readthedocs.io
8. scikit-learn documentation: https://scikit-learn.org
9. Project source code: https://github.com/NandKasar10/used-car-price-predictor
10. Dataset: [insert Kaggle dataset name and link]

13. Appendix

Appendix A: Project structure

```
used-car-price-predictor/
  dataset/car_data.csv
  notebooks/car_price_prediction.ipynb
  models/car_price_model.pkl
  app.py
  requirements.txt
  README.md
```

Appendix B: How to run

1. Install the libraries: pip install -r requirements.txt
2. Open the notebook in notebooks/ and run all cells. This creates models/car_price_model.pkl.
3. From the project folder, run: streamlit run app.py

Appendix C: Key code

Feature engineering:

```
df['brand_model'] = df['brand'] + ' ' + df['model']
df['car_age'] = 2026 - df['year']
counts = df['brand_model'].value_counts()
top_cars = counts[counts >= 15].index
df['brand_model'] = df['brand_model'].apply(lambda x: x if x in top_cars else
'Other')
```

Prediction in the web app:

```
input_encoded = pd.get_dummies(input_df, columns=['brand_model', 'fuel',
'seller_type', 'transmission'])
input_final = input_encoded.reindex(columns=model_columns, fill_value=0)
prediction = np.expm1(model.predict(input_final)[0])
```